

摘要

本报告介绍一套 MCP (Model Context Protocol) Server 综合评测系统及其在四个被测对象上的横向基准结果。系统从功能、性能、安全、LLM 兼容性、场景协同五个维度对 MCP Server 进行自动化评测，输出 A+ 至 D 的综合评级。与现有评测工具的主要差异在于后两个维度：LLM 兼容性维度引入真实大语言模型 (glm-4-plus) 阅读被测 Server 的工具目录并执行工具选择与参数填充，直接检验工具描述对模型的"可理解性"；场景协同维度通过多步骤状态传递工作流（写入、检索、删除、验证）检验工具间的行为一致性，并通过 LLM Agent 循环测试复合任务的自主规划能力。

评测系统在自研 Rust 语义基础设施 FedCtx 上发现两个真实缺陷——stdio 模式日志污染协议通道（违反 MCP 规范）、MCP 路径写操作绕过审计链（违反审计完整性契约）——均完成"发现、修复、复测验证"闭环，验证了评测方法的有效性。横向基准显示：四个被测 Server（自研 FedCtx、官方 filesystem、官方 memory、Python 参考实现）在标准套件上均达到 A+ 级；LLM 兼容性测试则揭示官方 filesystem server 存在 read_file 与 read_text_file 语义混淆问题（准确率 92.9%），表明该维度能够发现传统测试无法覆盖的工具设计缺陷。

1 系统概述

1.1 评测动机

MCP 正在成为 LLM Agent 与外部工具交互的事实标准协议，但 Server 实现质量参差不齐：协议合规性缺陷（如日志混入协议通道）、行为一致性缺陷（如审计遗漏）、工具描述对 LLM 不友好等问题普遍存在且难以被传统测试发现。现有 MCP 评测多停留在协议层功能验证与延迟统计，缺少对"LLM 能否正确使用这个 Server"这一核心问题的直接检验。

本系统的设计目标是回答三个层次的问题：

- 协议层：Server 是否正确实现了 MCP 规范，是否健壮、快速；
- 语义层：真实 LLM 阅读工具目录后能否选对工具、填对参数；
- 行为层：多工具协同的工作流中，状态传递与数据一致性是否可靠。

1.2 系统架构

系统为 Streamlit Web 应用（8 页面）加 Python 评测引擎的双层架构，SQLite 持久化全部测试记录。评测引擎支持 stdio 与 SSE 双 transport 连接被测 Server；LLM 相关维度通过 z-ai SDK 调用 glm-4-plus 模型。评分引擎将各维度结果加权汇总为 A+/A/B/C/D 五档评级（功能 40% + 性能 30% + 安全 30%），性能分由 P95 延迟映射（100ms 满分、5000ms 零分的线性插值），并支持 Markdown 报告导出。

1.3 五维评测体系

维度	用例数	检验目标	方法
功能	7	握手、工具列表、有效/无效参数、不存在工具、可选能力(resources/prompts)	标准 MCP 客户端协议交互

维度	用例数	检验目标	方法
性能	3	调用延迟(P95/P99)、并发吞吐(RPS)、连续调用稳定性	20 次延迟采样 + 1/5/10 并发压测
安全	5	SQL 注入、路径穿越、超长参数、特殊字符、超时容忍（均要求不崩溃）	对抗性输入注入
LLM 兼容	按工具数	工具选择准确率、参数 Schema 合规率	真实 LLM 两阶段评测 (glm-4-plus)
场景协同	6	多工具状态传递、CRUD 语义、审计一致性、持久化往返、Agent 复合任务	多步骤 workflow 断言链

表 1 五维评测体系总览

2 评测方法

2.1 标准测试套件

标准套件共 15 个用例（功能 FN-001 至 FN-007、性能 PF-001 至 PF-003、安全 SEC-001 至 SEC-005）。可选能力（resources、prompts）不支持时记为 skipped，不计入通过率分母，避免惩罚实现合法子集的 Server。

2.2 LLM 兼容性引擎

评测分两阶段。第一阶段，LLM 为被测 Server 的每个工具生成一条自然语言任务（要求任务只通过该工具完成，参数值在任务描述中给出）；第二阶段，LLM 在仅见工具目录与任务文本的条件下输出工具选择与参数 JSON，规则引擎校验工具名正确性与参数 Schema 合规性（含 JSON Schema 数组类型语法支持）。评分权重为工具选择 60% + 参数合规 40%。该方法将"工具描述质量"转化为可量化指标，能发现语义近似工具导致的模型混淆——这类缺陷对人类开发者不明显，但直接影响 Agent 的调用成功率。

2.3 场景化测试框架

场景由多个工具调用步骤组成，步骤间通过上下文传递状态（如第 1 步写入的数据第 2 步必须能读到），每步附断言函数，断言链断裂即终止后续步骤。内置 6 个场景覆盖向量/记忆/图谱生命周期、审计一致性、持久化往返，并含适配官方 server-memory 工具族的知识图谱场景。场景按被测 Server 实际拥有的工具自动匹配，缺失工具的场景记为 skipped（半通用设计）。

2.4 LLM Agent 循环

Agent 循环向 LLM 下发复合任务（如"记住医疗事实、验证可检索、删除、验证已删除"四步流程），LLM 在每轮输出下一步工具调用决策（工具名 + 参数 JSON），系统执行后将响应摘要回填至对话历史，循环至 LLM 输出完成标记或达到轮数上限。评估维度包括终态正确性（硬指标：循环结束后探测工具实际状态）、规划效率（实际步数）与循环安全性（轮数上限内收敛）。

3 横向基准结果

3.1 被测对象

Server	实现	工具数	说明
FedCtx (v0.8.0)	Rust + rmcp	14	自研联邦语义基础设施：HNSW 向量库、知识图谱、记忆引擎、SHA-256 审计链
filesystem (官方)	TypeScript	14	MCP 官方参考实现，文件系统读写与目录管理
memory (官方)	TypeScript	9	MCP 官方参考实现，基于知识图谱的持久记忆
test-echo	Python	3	平台内置参考实现，用于引擎自检

表 2 被测对象一览

3.2 总览

Server	标准套件	综合评级	LLM 兼容性	场景协同
FedCtx	15/15	A+ (100)	14/14 = 100%	5/5 适用场景通过
filesystem	13/15 (2 skip)	A+ (100)	13/14 = 92.9%	0/6 (工具族不匹配)
memory	14/15 (1 skip)	A+ (100)	9/9 = 100%	1/6 (SC-006 通过)
test-echo	12/15	A (88.6)	3/3 = 100%	-

表 3 五维评测总览。skipped 为可选能力 (resources/prompts) 或场景工具族不匹配，不计入失败。

3.3 性能

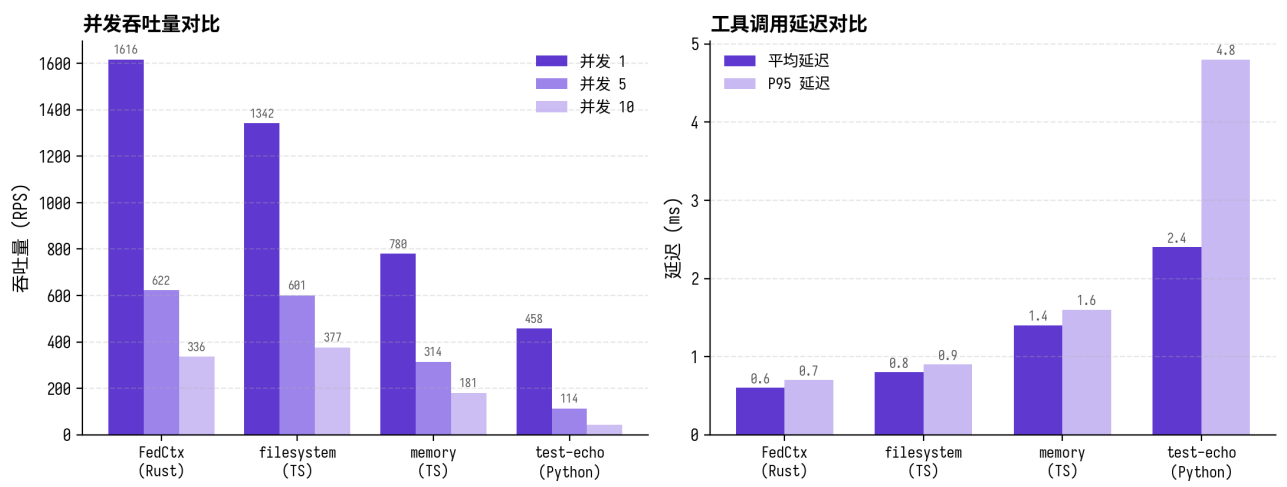


图 1 并发吞吐量（左）与调用延迟（右）对比。数据为 PF-001/PF-002 用例原始统计。

FedCtx (Rust) 以单并发 1616 RPS、平均延迟 0.6ms 居首；官方 filesystem (Node.js) 以 1342 RPS 紧随；官方 memory 因知识图谱遍历开销为 780 RPS；Python 参考实现 458 RPS 体现解释器开销基线。并发压力下 FedCtx 与 filesystem 衰减曲线接近（10 并发约为单并发的 21% 与 28%），显示 stdio JSON-RPC 通道序列化是主要瓶颈。

3.4 LLM 兼容性发现

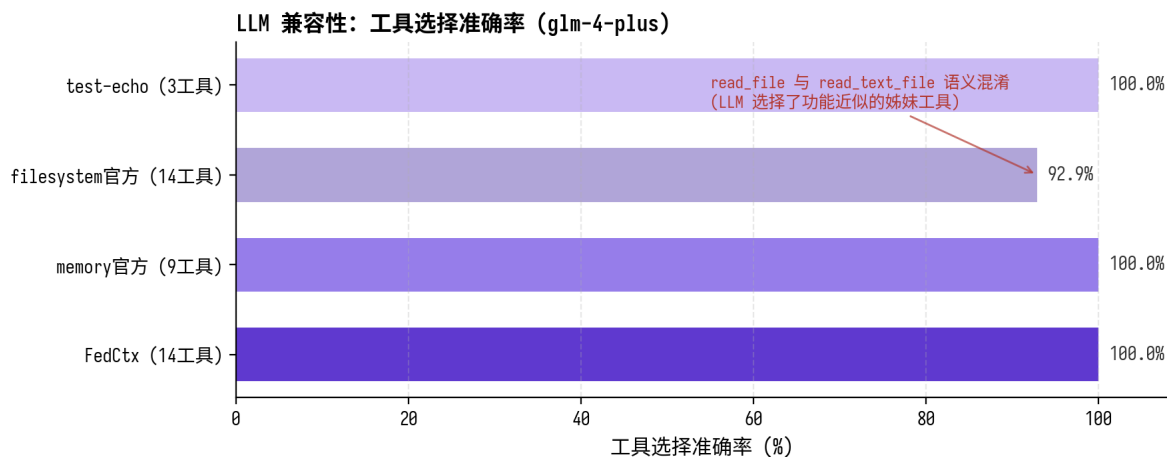


图 2 LLM 工具选择准确率。filesystem 官方实现出现 read_file 与 read_text_file 的语义混淆。

官方 filesystem server 的 read_file 与 read_text_file 功能高度近似（差异仅在于返回格式），LLM 在"读取配置文件"任务上选择了 read_text_file 而非预期工具。这并非模型能力不足——两个工具的描述不足以让模型建立确定性的选择边界。该发现说明：工具命名与描述的区分度本身就是接口设计质量的一部分，且只能通过真实模型评测发现。FedCtx 的 14 个工具（含语义接近的 memory_* 系列）全部通过，表明 rmcp 从 Rust doc comment 自动生成的工具描述具备充分区分度。

4 缺陷发现案例研究

评测系统在 FedCtx 上发现两个真实缺陷，均完成"发现、修复、复测验证"闭环，是评测方法有效性的直接证据。

4.1 案例一：stdio 日志污染协议通道

发现过程：FN-001 握手测试期间，MCP 客户端连续抛出 JSONRPC 消息解析失败异常。根因：FedCtx 的 tracing 日志（含 ANSI 颜色码）输出到 stdout，而 MCP 规范要求 stdio 模式下 stdout 仅承载 JSON-RPC 消息。宽松解析的客户端可跳过污染行，但严格实现的下游客户端会直接解析失败。

修复：初始化 tracing 时将 writer 定向至 stderr，stdio 模式下同时禁用 ANSI 颜色码（with_writer(std::io::stderr) + with_ansi(false)，commit e291186）。

验证：修复后完整套件复测，JSON-RPC 解析错误从每连接多次降为 0 次，15/15 用例通过。

4.2 案例二：MCP 写操作绕过审计链

发现过程：SC-004 审计一致性场景断言"写操作前后审计链条目数必须递增"，FedCtx 首次运行 5 到 5（未递增），场景失败。根因：insert_vector、add_graph_node、add_graph_edge 三个 MCP 工具直接写存储，未调用 audit.append，而 REST 与 gRPC 路径均有审计——审计链"记录每个向量/图谱操作"的契约被 MCP 路径破坏。

修复：为三个写工具补齐 audit.append 调用，图谱操作采用 Custom 事件类型（commit a52aa74）。

验证：复测 SC-004 通过（审计条目 5 到 7），审计链正确记录 vector_insert(1) node=mcp 与 custom(graph_node_add) node=mcp；全部 5 个场景通过。

4.3 闭环方法论

两个案例分别由不同维度捕获：案例一是功能用例执行期间的异常观察，案例二是场景断言的确定性失败。后者尤其体现了场景化测试的价值——它是唯一能检验"跨工具行为契约"的方法：审计一致性涉及写工具与审计查询工具的协同，单工具测试无法覆盖。发现、修复、复测的完整记录（含 commit 关联）使评测结果可审计、可追溯。

5 讨论与局限

- (1) LLM 兼容性评测依赖单一模型（glm-4-plus），模型间差异未覆盖；后续将接入多模型交叉验证，区分"描述歧义"与"模型能力不足"。
- (2) 场景套件的工具族绑定较强（向量/记忆/图谱），对工具族不匹配的 Server 只能给出 skipped 而非适配性评价；SC-006 的知识图谱场景已验证按工具族扩展场景的可行性。
- (3) 性能测试基于 stdio transport 的本地进程间通信，未覆盖 Streamable HTTP 远程部署场景的网络开销。
- (4) 四个被测对象均达 A+ 级，说明标准套件区分度有限；区分度主要来自 LLM 兼容性与场景维度，印证了引入语义层与行为层评测的必要性。

6 结论

本系统将 MCP Server 评测从协议层扩展到语义层（LLM 兼容性）与行为层（场景协同、Agent 复合任务），横向基准覆盖自研与官方实现共四个 Server。评测在 FedCtx 上发现的两个真实缺陷均完成修复验证闭环；在官方 filesystem server 上发现的工具语义混淆（92.9% 准确率）进一步表明，真实模型参与的评测能够暴露传统测试无法覆盖的接口设计问题。系统已开源（GitHub: dechang64/mcp-eval-platform），全部测试记录 SQLite 可追溯。